

Simulating Memory: Devices to Systems

ECE2.414 — Advanced Memory Circuits and Systems, Assignment 1

Hrishikesh Gawas

September 12, 2026

Question. *Should the 2 MB L2 cache in an accelerator be SRAM or STT-MRAM?* The question is carried through five tools, each working at a different level of abstraction: ngspice for the bitcell, CACTI for the SRAM array, NVSim for the STT-MRAM array, Ramulator 2.0 for the DRAM channel behind it, and gem5 for the effect on a running program. Each part's output becomes the next part's input. Stated accuracies are $\pm 5\%$ for ngspice, $\pm 10\%$ for CACTI and Ramulator, and $\pm 20\%$ for gem5, and these compound multiplicatively, so results are reported as ratios between configurations wherever possible.

Sources: <https://github.com/HrishikeshGawasIIITH/SRAM-vs-STT-MRAM>

1 Part A — ngspice: 6T SRAM read margin

1.1 Setup

A single 6T cell on the 45 nm PTM BSIM4 high-performance card. Pull-ups $W = 0.15 \mu\text{m}$, pull-downs $W = 0.20 \mu\text{m}$, access transistors $W_A = 0.16 \mu\text{m}$, all $L = 0.045 \mu\text{m}$. Each bitline carries $C_{BL} = 180 \text{ fF}$. Stored state is $Q = 0$, $\bar{Q} = 1$, set by `.ic v(q)=0` during the operating-point solve; the cross-coupled pair then holds $\bar{Q}$ at the rail by itself.

Bitlines are precharged through two PMOS devices ($W = 1.2 \mu\text{m}$) from V_{DD} to BL and BLB, with an equalising PMOS ($W = 0.6 \mu\text{m}$) across them. Precharge is active-low and releases at $t = 0.75 \text{ ns}$, leaving both lines floating at the rail. The wordline rises at $t = 1.0 \text{ ns}$ and ΔV is read at $t = 2.0 \text{ ns}$, giving a 1 ns development window. Wordline and precharge amplitudes are generated as unit pulses scaled by $v(V_{DD})$ so that they follow the supply through the Task 3 sweep. Transient step is 5 ps over 4 ns.

Average read current is inferred from the measured swing as $\bar{I}_{\text{read}} = C_{BL} \Delta V / \Delta t$.

Table 1: Part A summary at the four measured operating points.

Configuration	ΔV @2 ns (mV)	$\bar{I}_{\text{read}}$ (μA)	$\max v(Q)$ (mV)	as % V_{DD}	$v(Q)$ @4 ns (mV)
Task 1 — nominal, $W_A=0.16 \mu\text{m}$, 1.1 V, 27 °C	727	131	206	19 %	2.9
Task 2 — wide access, $W_A=0.24 \mu\text{m}$, 1.1 V, 27 °C	851	153	272	25 %	0.8
Task 4 — nominal, 1.1 V, 85 °C	579	104	221	20 %	15.2
Task 4 — nominal, 0.7 V, 85 °C	192	35	104	15 %	60.0

1.2 Task 1 — ΔV at 2 ns

At 1.1 V and 27 °C, $\Delta V(\text{BLB}, \text{BL}) = 727 \text{ mV}$, which is $10.5\times$ the 69 mV reference from Lecture 1. Fig. 1 shows why: BL does not hold a small differential but falls to 0.37 V by the sense point and 6 mV by 4 ns. The measurement therefore samples an almost complete discharge, not a developing signal.

The swing corresponds to 131 μA through a $0.16 \mu\text{m}$ device, about $0.8 \text{ mA}/\mu\text{m}$ — a normal on-current for a 45 nm high-performance NMOS at 1.1 V. Only $\approx 12 \text{ fF}$ is needed to reach 69 mV on 180 fF, which this cell supplies in under 100 ps. The same netlist on the low-power card gives 248 mV, $2.9\times$ smaller (Table 2). A 69 mV figure therefore corresponds to an earlier sense point, a heavier bitline,

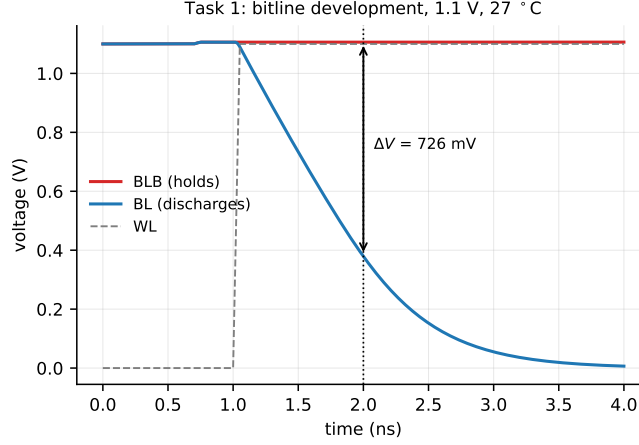


Figure 1: Bitline development at 1.1 V, 27 °C.

or a low-power device, rather than to this cell sensed at 2 ns. The quantity that carries forward is the cell current, from which ΔV follows for any C_{BL} and sense timing.

Table 2: Effect of the device model card, all else identical.

PTM model card	ΔV @2 ns (mV)	$\bar{I}_{\text{read}}$ (μA)	ΔV / 69 mV
45 nm HP (high performance)	727	131	$10.5\times$
45 nm LP (low power)	248	45	$3.6\times$

1.3 Task 2 — storage-node disturb

Q rises to 206 mV (19 % of V_{DD}) during the read: the precharged bitline injects current into the “0” node faster than the pull-down removes it. At $W_A = 0.24 \mu\text{m}$ the disturb grows 32 % while ΔV improves only 17 %. The access transistor sets both the read current and the disturb, so widening it helps one and hurts the other.

The cell does not flip at either width — the peak stays roughly a factor of two below $V_{DD}/2$ and Q returns to 2.9 mV by 4 ns. Sweeping W_A (Fig. 2) locates the upset boundary at $W_A \approx 0.51 \mu\text{m}$, i.e. a cell ratio of 0.39 against the 1.25 of the given sizing. The supplied cell has about $3.2\times$ margin in access width for this device card, bitline load and timing window.

Upset is detected from BLB rather than from $\max v(Q)$. Once BL discharges it pulls Q back down, so the peak approaches $V_{DD}/2$ without crossing it. BLB can only lose charge if $\bar{Q}$ was pulled down, which requires the cell to have flipped. BLB droop and the collapse of ΔV from 949 mV to 272 mV both appear between $W_A = 0.50$ and $0.52 \mu\text{m}$.

1.4 Task 3 — supply sweep

V_{DD} is stepped from 1.1 V to 0.6 V in 50 mV steps inside a `.control` loop, at both temperatures (Table 3, Fig. 3).

ΔV falls almost linearly with V_{DD} . The read is discharge-limited rather than current-limited across this range: BL is pulled nearly to ground at every supply, so ΔV follows the precharge level V_{DD} instead of the strongly non-linear $I_{on}(V_{DD})$.

ΔV does not reach the 25 mV sense-amp offset anywhere in the requested range. The minimum is 106 mV, at 0.6 V and 85 °C, still more than $4\times$ the offset. Extrapolating the four lowest points linearly puts the crossing near 0.50 V at 27 °C and 0.51 V at 85 °C. These are extrapolations, not simulated points, and both lie near where V_{DD} approaches $V_{th} + |V_{th,p}|$ and the linear trend must break down.

Read disturb also improves as the supply falls, from 19 % of V_{DD} at 1.1 V to 12 % at 0.6 V, because the access and pull-down devices weaken together while the 1 ns window is fixed.

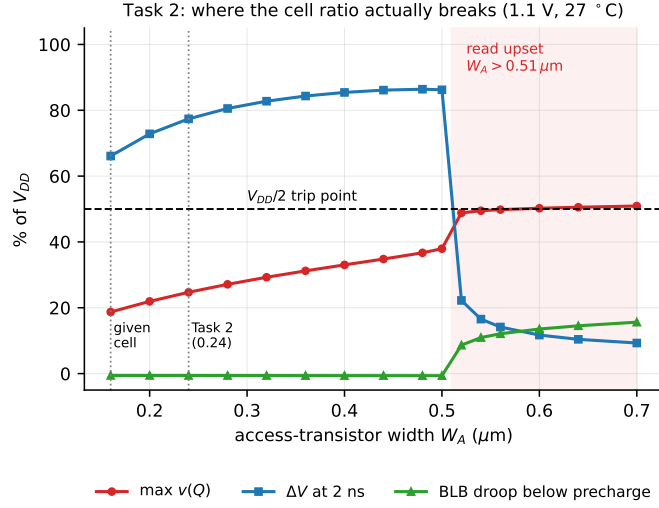


Figure 2: Access-width sweep at 1.1 V, 27 °C.

Table 3: Supply sweep at 27 and 85 °C.

V_{DD} (V)	$\Delta V(\text{BLB, BL})$ at 2 ns (mV)			max $v(Q)$ (% of V_{DD})	
	27 °C	85 °C	Δ	27 °C	85 °C
1.10	727	579	-148	19 %	20 %
1.05	671	531	-139	18 %	20 %
1.00	613	483	-130	18 %	19 %
0.95	554	434	-120	17 %	19 %
0.90	495	386	-109	17 %	18 %
0.85	435	337	-98	16 %	17 %
0.80	375	288	-87	15 %	17 %
0.75	315	240	-75	15 %	16 %
0.70	256	192	-63	14 %	15 %
0.65	198	147	-51	13 %	14 %
0.60	142	106	-36	12 %	13 %

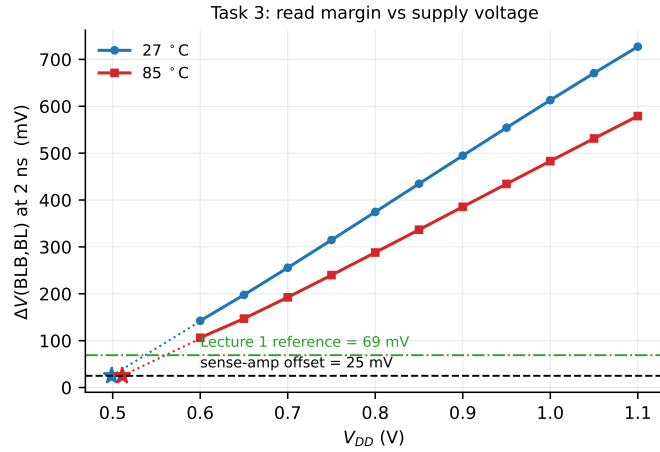


Figure 3: Read margin against supply. Dotted lines are linear extrapolations of the four lowest points.

1.5 Task 4 — 85 °C

At 1.1 V, raising the temperature to 85 °C drops ΔV from 727 mV to 579 mV, a loss of 148 mV (-20 %). At the 0.7 V, 85 °C corner $\Delta V = 192$ mV with the storage node disturbed to 15 % of V_{DD} .

Of the two quantities, ΔV moves far more. The sense-amp offset is set by threshold mismatch, is essentially temperature-independent, and changes by at most a few mV over 58 °C. ΔV falls -20 %, so the margin ratio drops from about 29× to 23×. The mechanism is mobility degradation, roughly $\mu \propto T^{-1.5}$, weakening the access transistor; this outweighs the opposing reduction in V_{th} with temperature at these overdrives.

Neither quantity sets the failure in this design. ΔV keeps more than 4× margin to the offset at the worst simulated point, and the disturb keeps more than 2.5× margin to the trip point, whereas Task 2 showed only 3.2× margin in access width. Read stability is the tighter constraint here.

Two limits of this analysis are worth stating. Every result is a single simulation at nominal process, so the σV_{th} spread across the 2^{24} cells of a 2 MB array — which is what actually decides whether the cell works — is not represented. And the answers depend on the device card chosen: the two PTM cards differ by 2.9× on the same netlist, and the high-performance card used here matches the `itrs-hp` setting used for CACTI in Part B.

Carried into Part B: cell read current ≈ 131 μ A (high-performance card), which sets the bitline delay CACTI reports and is used in the Task B4 hand check.

2 Part B — CACTI: 2 MB SRAM L2

2.1 Setup

CACTI 7 with the configuration given in the assignment: 2 MB capacity, 64 B blocks, 8-way associativity, one read/write port, 4 UCA banks, 45 nm, 350 K, `itrs-hp` for both the data-array cells and the periphery, cache type `cache`, and ED² as the optimisation target with design objective weights 0:0:0:100:0. Everything else is left at CACTI's defaults.

CACTI reports leakage per bank; with 4 banks the whole-cache figure is four times the reported value, and that is what is quoted here so it can be compared with NVSim in Part C.

Table 4: Part B baseline. This is the SRAM reference for Parts C–E.

Metric	2 MB SRAM L2, 45 nm
Access time (ns)	2.90
Cycle time (ns)	2.65
Dynamic read energy (pJ/access)	792.9
Dynamic write energy (pJ/access)	851.2
Leakage, whole cache (mW)	2251
Total area (mm ²)	10.66
Area efficiency (%)	54.3
Winning organisation (N _{dwl} /N _{dbl} /N _{spd})	4 / 2 / 1
Tag organisation (N _{twl} /N _{tbl} /N _{tspd})	2 / 2 / 2

2.2 Task 1 — baseline

Access time is 2.90 ns, dynamic read energy 793 pJ, whole-cache leakage 2251 mW and total area 10.66 mm², at an area efficiency of 54.3 %. The winning organisation is $N_{dwl}/N_{dbl}/N_{spd} = 4/2/1$. At a 2 GHz clock the access time is 6 cycles, which is the L2 hit latency used in Part E.

The delay breakdown is more informative than the total. Of the 2.90 ns, 47 % is H-tree wire delay into and out of the bank (1.36 ns combined), 0.54 ns is decoder and wordline, and only 14 % (0.407 ns) is the bitline itself. Sense amplification is negligible. At this capacity the access is dominated by interconnect, not by the array.

2.3 Task 2 — capacity sweep

Capacity is swept from 256 kB to 16 MB with every other parameter fixed (Table 5, Fig. 4).

Table 5: Capacity sweep, all other parameters fixed.

Capacity	$\log_2(\text{bytes})$	Access (ns)	Read (pJ)	Leak (mW)	Area (mm^2)
256 kB	18	2.42	658	361	4.86
512 kB	19	2.48	679	633	5.69
1 MB	20	2.63	717	1175	7.36
2 MB	21	2.90	793	2251	10.66
4 MB	22	3.53	930	4409	17.29
8 MB	23	4.47	1220	8735	34.30
16 MB	24	6.36	1742	16978	65.44

Amrutur & Horowitz’s “one gate delay per doubling” is not visible here. The rule predicts a constant increment of roughly one FO4 delay, about 20 ps at 45 nm. Measured, the increment grows steadily: 60 ps from 256 kB to 512 kB but 1.9 ns from 8 MB to 16 MB, a factor of about 30. A linear fit in $\log_2(\text{capacity})$ gives 0.597 ns per doubling and describes the curve poorly.

The reason follows from the breakdown above. The rule assumes each doubling adds one stage of decode logic, costing one gate delay, and holds while the array is logic-dominated. Past roughly 1 MB the H-tree dominates; doubling the capacity lengthens the wire by $\sqrt{2}$ in each dimension and RC wire delay grows with the square of length, so the increments compound rather than staying flat. The rule holds at the sub-MB capacities it was derived for, while a 2–16 MB L2 at 45 nm is wire-limited.

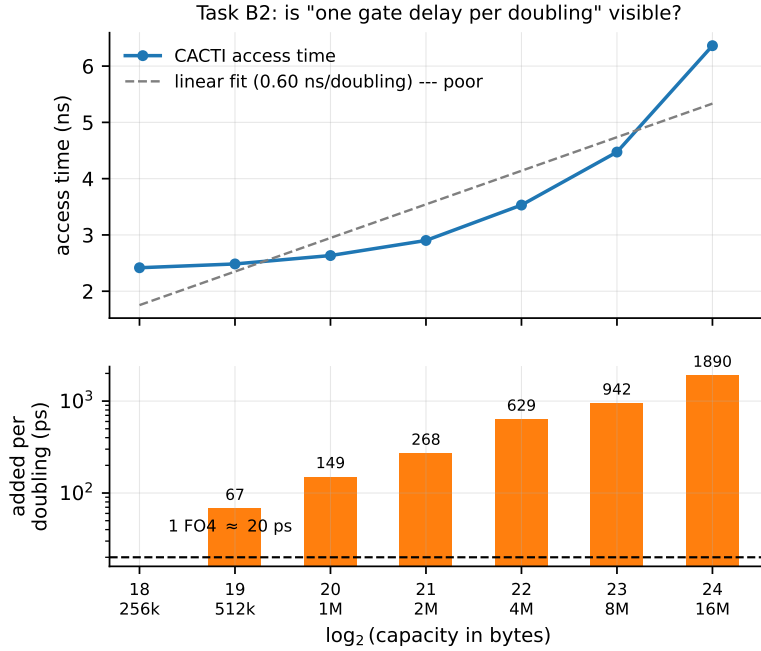


Figure 4: Access time against capacity (top) and delay added per doubling (bottom, log scale). A constant one-gate-delay increment would appear as flat bars.

2.4 Task 3 — objective

The same 2 MB cache is re-run three times with only the objective changed: pure delay (100:0:0:0:0 with ED optimisation off), pure area (0:0:0:0:100, likewise), and ED²P.

Delay and ED²P both select 4/2/1. Pure area selects $N_{dwl} = 2$ instead, trading $1.2\times$ the access time for 15 % less area (10.21 mm^2 to 8.65 mm^2).

N_{dwl} is the number of vertical cuts through the array, i.e. how many segments each wordline is divided into. Doubling it halves each wordline’s length and so roughly quarters its RC delay, but

Table 6: Same cache, three objectives.

Objective	Ndwl/Ndbl/Nspd	Access (ns)	Read (pJ)	Area (mm ²)
Pure delay (100:0:0:0:0)	4 / 2 / 1	2.89	898	10.21
Pure area (0:0:0:0:100)	2 / 2 / 1	3.36	797	8.65
ED ² P	4 / 2 / 1	2.90	793	10.66

every cut duplicates the decoder, wordline drivers and sense-amplifier strip along that boundary, none of which store any bits. The delay objective accepts that overhead because it scores only time; the area objective rejects it because it scores only area. The optimum is a property of the objective, not of the cache. The baseline used here is optimal for ED²P and is 15 % larger than necessary if area is the figure of merit — which matters in Part C, where area is the currency STT-MRAM is bought with.

2.5 Task 4 — bitline delay hand check

CACTI reports 0.407 ns of bitline delay for the Task 1 winner. The Lecture 4 estimate for a distributed RC line is $t = 0.38 rcL^2$. Using the winning subarray’s bitline length $L = 336 \mu\text{m}$, a minimum-pitch local wire with $r = 6.0 \Omega/\mu\text{m}$ (copper, $\rho = 2.2 \times 10^{-8} \Omega\text{m}$, $45 \text{ nm} \times 81 \text{ nm}$ cross-section) and $c = 0.40 \text{ fF}/\mu\text{m}$ including the cell drain junctions, the estimate is 0.104 ns. CACTI’s value is $3.9\times$ larger.

The two expressions answer different questions. $0.38 rcL^2$ is the step response of an RC line driven by an ideal voltage source, whereas a bitline is discharged by a single cell behaving as a current source — Part A measured that current at $131 \mu\text{A}$. The relevant delay is the time to develop the sense amplifier’s required differential, $C_{BL}\Delta V/I_{cell}$, not an RC settling time. CACTI accounts for the finite cell drive; the hand calculation assumes infinite drive and is therefore optimistic. Two smaller effects work the same way: CACTI includes the precharge and column-mux transistors loading the bitline, and it requires the full sense-amp offset to be developed rather than a 50 % crossing.

3 Part C — NVSim: 2 MB as STT-MRAM

3.1 Setup

NVSim at 2 MB, 8-way, 512-bit word, 45 nm, 350 K, HP device roadmap, H-tree routing, sequential cache access mode, read-EDP optimisation target. The array organisation and peripheral search are NVSim’s own; only the bitcell differs from Part B.

The cell file is the one specified in the assignment: MRAM, 54 F^2 cell area, aspect ratio 2.0, $R_{on}/R_{off} = 3/6 \text{ k}\Omega$, current read mode at $40 \mu\text{A}$, $145 \mu\text{W}$ read power, current set/reset at $200 \mu\text{A}$ for 10 ns, CMOS access device of width 6 F.

NVSim additionally requires four fields the assignment’s listing omits: `ReadVoltage`, `MinSenseVoltage`, `SetEnergy` and `ResetEnergy`. These are taken from NVSim’s shipped `sample_STTRAM.cell`, with `MinSenseVoltage` set to 25 mV so the sense-amp offset matches the one used in Part A.

Table 7: 2 MB L2 at 45 nm: SRAM (CACTI) against STT-MRAM (NVSim). SRAM leakage is the whole-cache figure.

Metric	SRAM (CACTI)	STT-MRAM (NVSim)	Ratio
Read latency	2.90	2.53	0.87×
Write latency	2.90	10.53	3.63×
Read energy	792.88	562.00	0.71×
Write energy	851.22	1133.00	1.33×
Leakage	2250.53	433.93	0.19×
Area	10.66	2.89	0.27×

3.2 Tasks 1–2 — comparison with the SRAM baseline

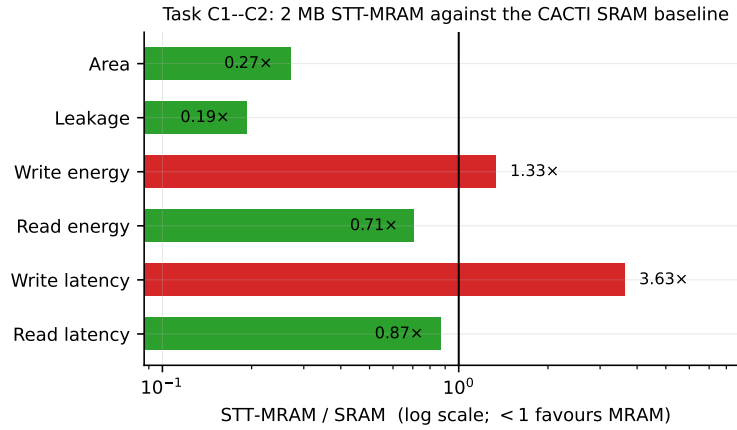


Figure 5: STT-MRAM relative to SRAM per metric, log scale. Values below 1 favour MRAM.

Two metrics improve by more than $3\times$ and two degrade.

Area and leakage improve. Area falls $3.7\times$ (10.66 mm^2 to 2.89 mm^2) and leakage $5.2\times$ (2251 mW to 434 mW). Both follow from the bitcell. A 6T SRAM cell is six transistors, roughly 140 F^2 , and retains its value through a cross-coupled pair permanently connected to the rail, which draws current continuously. An MTJ with one access transistor is 54 F^2 and retains its value in the magnetisation of the free layer, which draws none. The remaining 434 mW is peripheral — decoders, sense amplifiers, H-tree drivers — so the leakage ratio would improve further at larger capacities.

Write latency and write energy degrade. Writes take $3.6\times$ longer (2.90 ns to 10.53 ns) and cost $1.3\times$ more energy. This is switching physics, not periphery. Writing an SRAM cell means overpowering a latch, which takes about a gate delay. Writing an MTJ means passing enough current for long enough to reverse the free layer by spin-transfer torque, a thermally activated process whose switching probability depends on the overdrive above the critical current and on the pulse duration. The cell file specifies $200\text{ }\mu\text{A}$ for 10 ns , and that 10 ns pulse accounts for nearly all of the 10.53 ns write latency.

Read latency and read energy barely move ($0.87\times$ each, marginally in MRAM’s favour). This follows from Part B: 47% of the SRAM access is H-tree wire delay, which is unaffected by the bitcell, so changing the cell changes only the array-internal fraction. The comparison does cross two tools with different peripheral assumptions — NVSim is in sequential access mode here — so the supportable statement is that read latency is comparable, not that MRAM is 7% faster.

3.3 Task 3 — TMR ratio

Re-running with $R_{on} = 4\text{ k}\Omega$ and $R_{off} = 12\text{ k}\Omega$ (TMR 3:1 instead of 2:1) changes read latency from 2.53 ns to 2.54 ns ($+0.3\%$), read energy from 562 pJ to 561 pJ , and area from 2.89 mm^2 to 2.88 mm^2 . Sweeping the ratio from 1.5:1 to 6:1 moves read latency by 0.5% in total and leaves the other outputs unchanged. Read latency is the metric that moves most, by less than the tool’s error bar.

The cell file uses `ReadMode: current` with an explicit `ReadCurrent` and `MinSenseVoltage`, so NVSim sizes the sensing path from those two values and the resistance ratio enters only weakly. That is consistent with what TMR physically provides: sense margin, meaning the separation between the “0” and “1” distributions relative to their spread from MTJ process variation and reference-cell offset. TMR buys read reliability — bit error rate, and therefore yield and ECC overhead — rather than read speed or read energy. A deterministic array model has no variation distribution to integrate over and so cannot price it. A TMR headline needs a statistical model before it becomes an architectural number.

3.4 Task 4 — write current and cell area

Halving `ResetCurrent` from 200 to 100 μA changes area by 0 %: 2.89 mm^2 to 2.89 mm^2 . Sweeping the write current over a wider range shows why (Fig. 6).

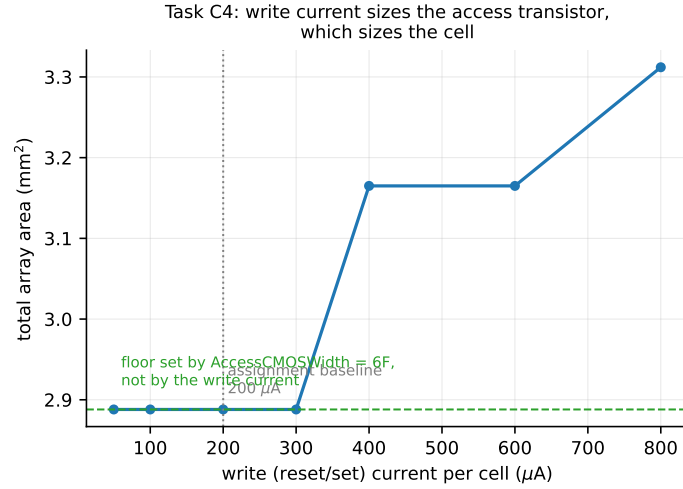


Figure 6: Array area against per-cell write current.

Area is flat at 2.89 mm^2 from 50 to 300 μA , steps up at 400 μA , and reaches 3.31 mm^2 (+15 %) at 800 μA . The coupling is:

write current $\rightarrow$ access-transistor width $\rightarrow$ cell area $\rightarrow$ array density

The access transistor must supply the full write current and an MOS device delivers current in proportion to its width, so $W \propto I_{\text{write}}$. In an STT-MRAM cell the MTJ sits above the metal stack and consumes no silicon area of its own, so the cell footprint is the access-transistor footprint. Below 400 μA the width demanded by the write current is smaller than the `AccessCMOSWidth` of 6 F declared in the cell file, so the declared width is what binds and the area does not move. Halving from 200 to 100 μA therefore changes nothing, because 200 was already on the flat part. Above the knee the current binds and area tracks it.

This matters for Part E. The $3.7\times$ density advantage is what allows 7.4 MB of STT-MRAM to fit in the SRAM’s 10.66 mm^2 , and that advantage is a property of the write current rather than of the MTJ. A device requiring 800 μA to switch reliably returns 15 % of the density it was chosen for. Reducing switching current is therefore worth more at the array level than raising TMR: one moves the metric the technology is selected for, the other moves a metric this class of tool cannot resolve.

4 Part D — Ramulator 2.0: L2 miss stream on DDR4

4.1 Setup

DDR4-3200AA, 8 Gb x8 organisation, one channel, two ranks, generic controller with an open-row policy, FR-FCFS scheduling, all-bank refresh and RoBaRaCoCh address mapping. Frontend clock ratio 8, memory clock ratio 3. The relevant timings are $t_{\text{RCD}} = 22$, $t_{\text{RP}} = 22$, $t_{\text{RAS}} = 52$, $t_{\text{CL}} = 22$ and $t_{\text{CCD}} = 4$ (short) / 8 (long) cycles, with $t_{\text{CK}} = 0.625$ ns.

The trace is 57313 reads and 5708 writes recorded on the memory-side port of the Part E baseline L2 (2 MB, 8-way), running `bfs` over a 65 k-node Kronecker graph. The graph is built beforehand and loaded from file, so the recorded region is the traversal itself; generating the graph inside the simulator would fill the trace with cold-miss streaming from a compute-bound phase whose working set is small enough to sit in L1. Evictions that carry no data to DRAM (`CleanEvict`) are excluded; writebacks are kept.

Average read latency is computed as total accumulated read latency divided by the number of read requests the memory system reports. This is a saturated open-loop replay — the frontend injects a request in every cycle it can, with no dependent instruction to stall it — so absolute latencies are dominated by queueing rather than by DRAM timing, and only ratios between configurations are meaningful.

Table 8: Four configurations on the same trace. Latencies in memory cycles.

Configuration	Memory cycles	Avg read (cycles)	(ns)	Row-buffer hit rate	ACT commands
FR-FCFS, RoBaRaCoCh, 1 ch (baseline)	386761	867	542	90.5 %	5954
FCFS, RoBaRaCoCh, 1 ch	732818	1516	947	90.1 %	6195
FR-FCFS, ChRaBaRoCo, 1 ch	881951	1853	1158	91.3 %	5463
FR-FCFS, RoBaRaCoCh, 2 ch	170205	655	410	92.0 %	2515

Part D: scheduler, address mapping and channel count

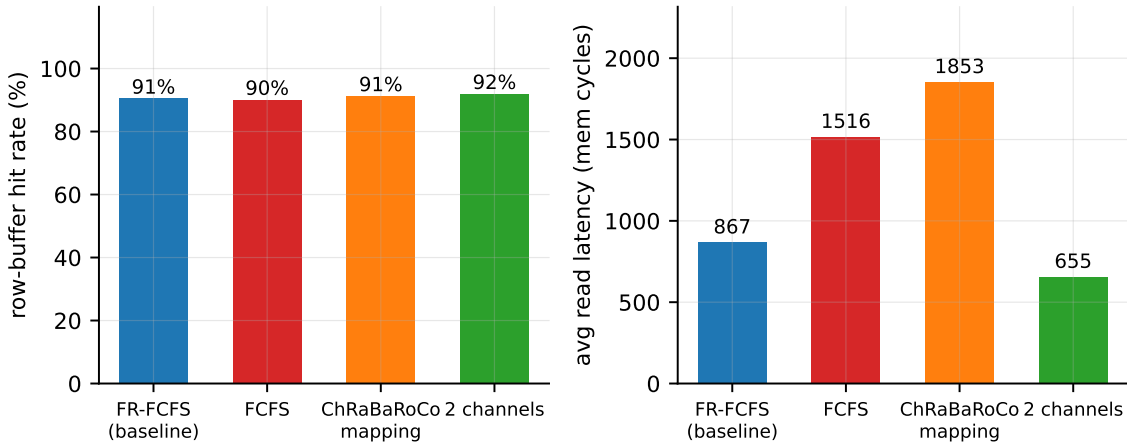


Figure 7: Row-buffer hit rate and average read latency across the four configurations.

4.2 Task 1 — baseline

FR-FCFS on one channel completes the trace in 386761 memory cycles with an average read latency of 867 cycles (542 ns) and a row-buffer hit rate of 90.5 %.

4.3 Task 2 — FCFS

Switching to strict first-come-first-served costs $1.89\times$ in total cycles (386761 to 732818) and $1.75\times$ in read latency (867 to 1516 cycles, +649 cycles per read).

The row-buffer hit rate does not collapse: it moves from 90.5 % to 90.1 %, and the activate count changes only from 5954 to 6195. So the usual explanation — that FCFS loses row hits and pays $t_{RP} + t_{RCD} = 44$ cycles for each one — does not apply here, since that would require thousands of additional ACT commands and only 4 % more appear.

What is lost is bank-level parallelism. A BFS miss stream over a CSR graph has high spatial locality, because consecutive misses walk an edge list, so most accesses find an open row regardless of the order they are serviced in. The row hits come from the access pattern, not from the scheduler. What FR-FCFS provides is the freedom to let a request to an idle bank overtake one whose bank is still occupied for t_{RAS}/t_{RC} . FCFS forbids that, so the oldest request blocks everything behind it including requests to banks that are free. The +649 cycles per read is queueing behind busy banks.

4.4 Task 3 — address mapping

Changing the mapping from RoBaRaCoCh to ChRaBaRoCo places the row bits below the bank bits in significance, so a sequential address sweep walks every column of a row and every row of a bank before reaching the next bank. This costs $2.28\times$ in total cycles, with a read latency of 1853 cycles — the worst of the four configurations, and worse than FCFS.

Row-buffer behaviour again does not explain it. The hit rate is higher than the baseline (91.3% against 90.5%) and there are fewer activates (5463 against 5954). The mapping is good for row locality and poor for bank parallelism, because the whole access stream is funnelled into one bank at a time while the remaining fifteen sit idle. The configuration with the best row-buffer hit rate is the slowest, so hit rate on its own is not a useful proxy for DRAM performance.

4.5 Task 4 — channel count

Doubling to two channels improves read latency by 24% (867 to 655 cycles) and more than doubles throughput ($2.27\times$). Extending the sweep gives $4.65\times$ at four channels and $8.36\times$ at eight, with latency levelling off at 536 and 535 cycles.

Latency therefore does not “barely improve” on this trace, and the binding parameter can be identified from the output. It is not t_{RCD} or t_{RP} : with a 90.5% row-hit rate only about one access in ten activates at all, and the ACT count (5954 against 57313 reads) shows this directly — if activation were the limit, adding channels would not help, because each channel would still pay t_{RCD} per access. The same argument rules out t_{RAS} . The limit at one channel is t_{CCD} : with nine accesses in ten being column reads to an already-open row, the constraint is how fast column commands can be issued onto a single shared data bus, $t_{CCD_S} = 4$ cycles across bank groups and $t_{CCD_L} = 8$ within one. Each additional channel brings its own bus and its own t_{CCD} budget, which is why throughput scales nearly linearly to four channels.

As a general diagnostic: compare the ACT count with the access count to test t_{RCD} and t_{RP} , and if activations are rare, compare the achieved reads-per-cycle against the t_{CCD} -implied bus ceiling. The flattening between four and eight channels is not a DRAM limit — the replay cannot inject quickly enough to keep eight buses busy.

The miss traffic does not fit comfortably in one DDR4 channel. The superlinear $2.27\times$ from the second channel only occurs if the first was saturated.

5 Part E — gem5: full-system effect

5.1 Setup

Single X86 core at 2 GHz in syscall-emulation mode, 32 kB 8-way L1I and L1D at 2-cycle latency, 8-way L2, DDR4-2400 8x8 main memory, 4 GB address space. Two GAPBS kernels, `bfs` and `sssp`, over a 65 k-node Kronecker graph (7.8 MB CSR) built beforehand and loaded from file, each run to completion. Loading a prepared graph keeps the simulated region in the traversal rather than in the graph generator, whose working set is small enough that the L2 is never exercised.

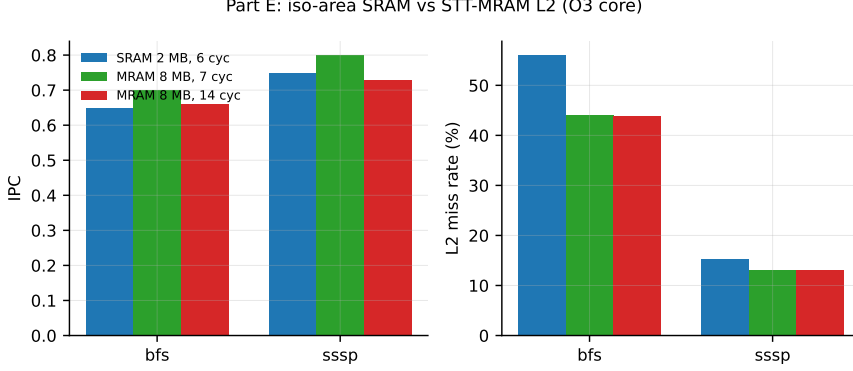
L2 capacities and hit latencies come from Parts B and C rather than from assumed values:

- **SRAM, 2 MB** — CACTI access time 2.90 ns, so 6 cycles at 2 GHz, in 10.66 mm^2 .
- **STT-MRAM, 8 MB** — NVSim hit latency 3.21 ns, so 7 cycles, in 10.32 mm^2 .

8 MB is the iso-area capacity: NVSim fits it in 10.32 mm^2 against CACTI’s 10.66 mm^2 for 2 MB of SRAM, so the swap is $4\times$ the capacity in the same silicon. The measured read penalty is one cycle, not the factor of two that a slower bitcell might suggest, because 47% of the access is H-tree wire delay and is unaffected by the cell. Since the whole comparison turns on that single number, a third configuration re-runs the MRAM at 14 cycles as a sensitivity check.

Table 9: Out-of-order core. `simTime` is simulated wall-clock.

Kernel	L2 configuration	IPC	L2 miss rate	simTime (ms)
bfs	SRAM 2 MB, 6 cyc	0.648	56.0 %	4.62
bfs	MRAM 8 MB, 7 cyc	0.701	44.0 %	4.27
bfs	MRAM 8 MB, 14 cyc	0.661	43.9 %	4.53
sssp	SRAM 2 MB, 6 cyc	0.749	15.3 %	21.96
sssp	MRAM 8 MB, 7 cyc	0.799	13.1 %	20.58
sssp	MRAM 8 MB, 14 cyc	0.728	13.1 %	22.59

**Figure 8:** IPC and L2 miss rate for the three L2 configurations.

5.2 Tasks 1–2 — which kernel benefits

At the measured latencies the MRAM L2 is faster on both kernels: $1.081\times$ on `bfs` and $1.067\times$ on `sssp`. At the pessimistic 14-cycle latency `bfs` still gains ($1.019\times$) while `sssp` loses ($0.972\times$).

The deciding property is the baseline miss rate. `bfs` misses 56.0 % of L2 accesses at 2 MB; its frontier and CSR working set do not fit, so $4\times$ the capacity converts into fewer DRAM accesses (56.0 % to 44.0 %, a 22 % reduction in absolute misses). `sssp` misses only 15.3 % at 2 MB, because its hot working set — the delta-stepping buckets and the distance array — largely fits, so extra capacity removes far less (14 %) while the added hit latency is charged on the $\sim 85\%$ of accesses that already hit.

Capacity pays off on misses and latency is a tax on hits, so the ratio of hits to misses determines the outcome. A kernel that is capacity-bound at the baseline size gains; a kernel that already fits pays the tax with little left to recover.

5.3 Task 3 — in-order core

Re-running with the in-order timing CPU reduces the advantage on both kernels: `bfs` from $1.081\times$ to $1.035\times$, and `sssp` from $1.067\times$ to $1.016\times$.

Per-miss accounting explains this. On `bfs`, each avoided L2 miss is worth 99 cycles to the in-order core but only 47 cycles to the out-of-order core. The in-order machine has one miss outstanding at a time, so an avoided miss returns a nearly fully exposed DRAM round trip. The out-of-order core keeps several misses in flight through its MSHRs, so misses overlap and any individual one sits only about half on the critical path. Out-of-order execution was already hiding roughly half the cost of every L2 miss, which is also why the extra hit latency of the MRAM costs so little there.

The relative advantage is nevertheless larger on the out-of-order core, because its baseline is much shorter: `bfs` finishes in 9.24 M cycles against 25.7 M in order, so the same absolute saving is a larger fraction of the run. Memory-level parallelism compresses the absolute benefit of a larger cache and amplifies its relative effect.

5.4 Task 4 — conclusion

(i) SRAM or STT-MRAM? On this evidence, STT-MRAM as an 8 MB iso-area replacement, though the margin is small and the case rests on capacity rather than on the device being intrinsically better. The chain is as follows. Part A measured 131 μA of read current per 6T cell and confirmed the cell reads correctly at 0.7 V and 85 °C with 7.7 $\times$ margin to the sense-amp offset, so the SRAM baseline is grounded in a device simulation rather than assumed. Part B sized that baseline at 2.90 ns, 793 pJ per read, 2251 mW of leakage and 10.66 mm², and showed that 47 % of the access is wire rather than array. Part C placed 8 MB of STT-MRAM in the same area with 5.2 $\times$ less leakage and 3.7 $\times$ the density, at 3.6 $\times$ the write latency. Because the access is wire-dominated, the read penalty for that swap is one cycle. Part D showed the resulting miss stream needs more than one DDR4 channel, with two channels giving 2.27 $\times$. Part E converts this into 1.081 $\times$ on **bfs** and 1.067 $\times$ on **sssp**, plus a standing leakage saving that no IPC figure captures.

Stated precisely: 8 MB of STT-MRAM is 6–8 % faster than 2 MB of SRAM on **bfs** and **sssp**, in the same area, given these device numbers, this cell file, an FR-FCFS scheduler and an out-of-order core — and it is 3 % slower on **sssp** if the hit-latency penalty is 7 cycles rather than the 1 measured here. It is a capacity argument with a leakage benefit, and it is sensitive to write-heavy workloads that were not run.

(ii) Three assumptions to attack first. *A deterministic corner is not a yield.* Every Part A number is one simulation at one process point. A 2 MB array holds 2²⁴ cells, and what decides whether the cell works is the σV_{th} tail across all of them, roughly 6 σ of per-cell margin. The 7.7 $\times$ margin at the worst corner and 3.2 $\times$ in access width are both single-point results, and Part C’s TMR sweep moved nothing precisely because NVSim carries no variation model with which to price sense margin. The reliability case is unmodelled throughout.

The cell file. The density advantage rests on **CellArea** of 54 F² and **AccessCMOSWidth** of 6 F, both asserted rather than derived. Task C4 showed area is flat in write current below 400 μA and then steps up, so a device needing 800 μA to switch reliably returns 15 % of the density the technology was chosen for. The 4 $\times$ capacity on which the Part E result depends is one line of a text file.

Workload coverage. Both kernels run a 65 k-node graph to completion on one core, and STT-MRAM’s weakest property by a wide margin is writing (3.6 $\times$ latency, 1.3 $\times$ energy), which a read-dominated graph traversal barely exercises. A write-heavy or multi-core workload could remove the 6–8 % advantage entirely. The cross-tool comparison deserves the same scrutiny: CACTI’s access time and NVSim’s hit latency rest on different peripheral assumptions, so the one-cycle penalty that makes this argument work is a difference between two models rather than a measurement of two fabricated caches.